

Java Cheat Sheet

String | StringBuilder | StringBuffer

1. Quick Comparison

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Performance	Slow for repeated changes	Fastest	Slower than StringBuilder
Since	JDK 1.0	JDK 1.5	JDK 1.0
Storage	String pool / heap	Heap	Heap
equals()	Compares content	Compares reference	Compares reference
Use when	Fixed text, keys, constants	Single-threaded string building	Multi-threaded string building

Rule of thumb: String by default, StringBuilder for loops and concatenation, StringBuffer only when several threads share the same buffer.

2. String

Creation

```
String s1 = "hello";           // string pool
String s2 = new String("hello"); // heap (new object)
String s3 = String.valueOf(123); // "123"
char[] arr = {'a', 'b'};
String s4 = new String(arr);    // "ab"
```

Common methods

Method	Example	Result
length()	"abc".length()	3
charAt(i)	"abc".charAt(1)	'b'
substring(b)	"hello".substring(2)	"llo"
substring(b, e)	"hello".substring(1, 3)	"el" (end exclusive)
indexOf(x)	"hello".indexOf('l')	2
lastIndexOf(x)	"hello".lastIndexOf('l')	3
contains(s)	"hello".contains("ell")	true
startsWith / endsWith	"hello".startsWith("he")	true
equals(s)	"a".equals("a")	true
equalsIgnoreCase(s)	"A".equalsIgnoreCase("a")	true
compareTo(s)	"a".compareTo("b")	-1
toUpperCase / toLowerCase	"Hi".toUpperCase()	"HI"
trim()	" hi ".trim()	"hi"
strip() (Java 11)	" hi ".strip()	"hi" (Unicode-aware)
replace(a, b)	"aab".replace('a', 'x')	"xxb"
replaceAll(regex, s)	"a1b2".replaceAll("\\d", "")	"ab"
split(regex)	"a,b,c".split(",")	["a", "b", "c"]
isEmpty()	"".isEmpty()	true
isBlank() (Java 11)	" ".isBlank()	true
concat(s)	"a".concat("b")	"ab"
toCharArray()	"abc".toCharArray()	['a', 'b', 'c']
String.join(sep, ...)	String.join("-", "a", "b")	"a-b"
repeat(n) (Java 11)	"ab".repeat(2)	"abab"
String.format(...)	String.format("%d-%s", 1, "x")	"1-x"

Method	Example	Result
intern()	s2.intern()	pooled reference
chars()	"abc".chars()	IntStream
lines() (Java 11)	"a\nb".lines()	Stream<String>

Key points

- Every "modification" returns a **new** String; the original is unchanged.
- == compares references, .equals() compares content.
- Literals with the same value share one object in the string pool.
- Text blocks (Java 15+) use """ for multi-line strings.

```
String a = "hi";
String b = "hi";
String c = new String("hi");
a == b;           // true (same pool object)
a == c;           // false (different objects)
a.equals(c);      // true
a == c.intern();  // true
```

3. StringBuilder

Creation

```
StringBuilder sb1 = new StringBuilder();           // capacity 16
StringBuilder sb2 = new StringBuilder("hello");    // capacity = 16 + length
StringBuilder sb3 = new StringBuilder(100);        // custom capacity
```

Common methods

Method	Description	Example
append(x)	Add to end	sb.append("a").append(1)
insert(i, x)	Insert at index	sb.insert(0, "Hi ")
delete(s, e)	Remove range (end exclusive)	sb.delete(0, 2)
deleteCharAt(i)	Remove one char	sb.deleteCharAt(0)
replace(s, e, str)	Replace range	sb.replace(0, 2, "XY")
reverse()	Reverse in place	sb.reverse()
setCharAt(i, c)	Change one char	sb.setCharAt(0, 'H')
setLength(n)	Truncate or pad with \0	sb.setLength(0) clears it
length()	Number of chars	sb.length()
capacity()	Current buffer size	sb.capacity()
ensureCapacity(n)	Reserve space	sb.ensureCapacity(500)
trimToSize()	Shrink capacity to length	sb.trimToSize()
charAt(i)	Get char	sb.charAt(0)
indexOf(s)	Find substring	sb.indexOf("lo")
substring(s, e)	Returns a String	sb.substring(1, 3)
toString()	Convert to String	sb.toString()
isEmpty() (Java 15+)	Check for empty	sb.isEmpty()
compareTo(sb2) (Java 11+)	Compare content	sb.compareTo(sb2)

Example

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) {
    sb.append(i).append(",");
}
sb.setLength(sb.length() - 1); // remove trailing comma
System.out.println(sb);       // 0,1,2,3,4
```

Capacity growth: when full, new capacity = (old + 1) * 2.

4. StringBuffer

API is **identical to StringBuilder**, but every method is **synchronized**.

```
StringBuffer sbf = new StringBuffer("Hello");
sbf.append(" World");
sbf.insert(0, ">> ");
sbf.reverse();
System.out.println(sbf.toString());
```

Use it only when several threads modify the **same** buffer. Otherwise prefer `StringBuilder`, since the synchronization overhead is wasted.

5. Conversions

```
// String -> StringBuilder / StringBuffer
StringBuilder sb = new StringBuilder(str);
StringBuffer sbf = new StringBuffer(str);

// StringBuilder / StringBuffer -> String
String s1 = sb.toString();
String s2 = sbf.toString();

// StringBuilder <-> StringBuffer (via CharSequence constructor)
StringBuffer sbf2 = new StringBuffer(sb);
StringBuilder sb2 = new StringBuilder(sbf);
```

6. Performance: why not + in loops

```
// Bad: new String each iteration -> O(n^2)
String s = "";
for (int i = 0; i < 10000; i++) s += i;

// Good: O(n)
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10000; i++) sb.append(i);
String result = sb.toString();
```

A single-expression `"a" + b + "c"` is compiled efficiently, so `+` is fine outside loops.

7. Common Interview Traps

- **equals()**: `StringBuilder` and `StringBuffer` don't override `equals()` or `hashCode()`, so `sb1.equals(sb2)` is false even with the same text. Use `sb1.toString().equals(sb2.toString())` or `compareTo` (Java 11+).
- String is **final and immutable**, making it safe for `HashMap` keys, security-sensitive values and class loading.
- Passwords are better stored as `char []` than `String`, because the array can be wiped after use.
- `substring()` since Java 7u6 copies the characters, so no memory leak from a shared array.
- `new String("a")` can create up to **2 objects** (one in the pool, one on the heap).
- `reverse()` exists on `StringBuilder/StringBuffer` (mutates in place), not on `String`.

8. Decision Guide

Situation	Choose
Constant or rarely changing text	String
Building or modifying text in a loop	StringBuilder
Shared across threads, mutated concurrently	StringBuffer
Joining a collection with a delimiter	String.join() or StringJoiner
Formatting output	String.format() or formatted()